```python
import logging
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
import tempfile
import os
import psycopg2
from pgvector.psycopg2 import register_vector
from dotenv import load_dotenv
from pydantic import BaseModel
from typing import Optional, List
from agent import run_resume_agent, extract_pdf, embed_text
import shutil
import numpy as np
import re
from contextlib import contextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()   # ✅ DEFINE FIRST

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)





# --------------------------------
# LOGGING
# --------------------------------
# logging.basicConfig(level=logging.ERROR)
# logger = logging.getLogger("resume_api")
logging.basicConfig(
    level=logging.ERROR,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
)
logger = logging.getLogger("resume_api")
```

```python
load_dotenv()

# --------------------------------
# DATABASE
# --------------------------------
def get_connection():
    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            port=os.getenv("DB_PORT"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD")
        )
        register_vector(conn)
        return conn
    except Exception as e:
        logger.error(e, exc_info=True)
        return None

# FIX: make it a real context manager
@contextmanager
def get_cursor():
    conn = get_connection()
    if not conn:
        raise HTTPException(status_code=500, detail="DB connection failed")
    cur = conn.cursor()
    try:
        yield cur
        conn.commit()
    finally:
        cur.close()
        conn.close()

# --------------------------------
# HELPERS
# --------------------------------
def safe_extract_pdf(path: str) -> str:
    try:
        text = extract_pdf(path)
        return text.encode("utf-8", errors="ignore").decode()
    except Exception:
        return ""
```

```
def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    if np.linalg.norm(a) == 0 or np.linalg.norm(b) == 0:
        return 0.0
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def chunk_text_simple(text, size=500):
    words = text.split()
    return [" ".join(words[i:i+size]) for i in range(0, len(words), size)]
# ----------------------------
# SAFE PDF TEXT EXTRACTION
# ----------------------------
def safe_extract_pdf(file_path: str) -> str:
    """
    Extract text from PDF safely, ignoring UTF-8 decode errors.
    """
    try:
        text = extract_pdf(file_path)
        if not text:
            return ""
        return text.encode("utf-8", errors="ignore").decode("utf-8",
errors="ignore")
    except Exception as e:
        logger.error(f"PDF extraction failed for {file_path}: {e}",
exc_info=True)
        return ""
 # ----------------------------
# SAFE PDF TEXT EXTRACTION
# ----------------------------
def safe_extract_pdf(file_path: str) -> str:
    """
    Extract text from PDF safely, ignoring UTF-8 decode errors.
    """
    try:
        text = extract_pdf(file_path)
        if not text:
            return ""
        return text.encode("utf-8", errors="ignore").decode("utf-8",
errors="ignore")
    except Exception as e:
        logger.error(f"PDF extraction failed for {file_path}: {e}",
exc_info=True)
        return ""
```

```python
# ----------------------------
# UTILITY FUNCTIONS
# ----------------------------
def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    if np.linalg.norm(a) == 0 or np.linalg.norm(b) == 0:
        return 0.0
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# def chunk_text_simple(text: str, chunk_size: int = 500):
#     words = text.split()
#     return [" ".join(words[i:i+chunk_size]) for i in range(0, len(words),
chunk_size)]
# # --------------------------------
# TEST DATABASE
# --------------------------------
@app.get("/test-db")
def test_db():
    try:
        conn = get_connection()
        if conn:
            conn.close()
            return {"status": "success"}
        return {"status": "failed", "reason": "Unable to connect to DB"}
    except Exception as e:
        logger.error("Error in /test-db endpoint: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))



# --------------------------------
# MODELS
# --------------------------------
class Job(BaseModel):
    job_role: str

class Skill(BaseModel):
    skill_name: str
    job_id: int

class AgentResult(BaseModel):
    filename: str
    similarity_score: float
```

```
    match: str
    agent_output: str
    chunks_saved: int
    top_chunks: List[dict]
    skills: List[str]

class BestMatch(BaseModel):
    job_id: int
    job_role: str
    skills: List[str]

class AutoMatchResult(BaseModel):
    filename: str
    best_match: BestMatch
    similarity_score: float
    match: str
    agent_output: str
# --------------------------------
# JOB ENDPOINTS
# --------------------------------
@app.get("/jobs")
def list_jobs(job_role: Optional[str] = None):
    logger.debug("List jobs request received.")
    conn = get_connection()
    if not conn:
        raise HTTPException(status_code=500, detail="Database connection failed")

    try:
        cur = conn.cursor()
        if job_role:
            logger.debug(f"Filtering jobs with role: {job_role}")
            cur.execute(
                '''
                SELECT "ID", "Job Role", "Skills"
                FROM "RESUME"."Job_descriptions"
                WHERE "Job Role" ILIKE %s
                ''',
                (f"%{job_role}%",)
            )
        else:
            logger.debug("Fetching all jobs.")
            cur.execute('SELECT "ID", "Job Role", "Skills" FROM
"RESUME"."Job_descriptions"')
```

```
        rows = cur.fetchall()
        logger.info(f"Fetched {len(rows)} jobs.")
        return [{"ID": r[0], "Job Role": r[1], "Skills": r[2]} for r in rows]

    except Exception as e:
        logger.error("Failed to fetch jobs: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail="Error fetching job records")
    finally:
        cur.close()
        conn.close()



@app.post("/jobs-add")
def create_jobcdd(job: Job):
    logger.debug(f"Create job request received: {job.job_role}")
    conn = get_connection()
    if not conn:
        raise HTTPException(status_code=500, detail="Database connection failed")

    try:
        cur = conn.cursor()
        cur.execute(
            '''
            INSERT INTO "RESUME"."Job_descriptions" ("Job Role")
            VALUES (%s) RETURNING "ID";
            ''',
            (job.job_role,)
        )
        job_id = cur.fetchone()[0]
        conn.commit()
        logger.info(f"Job created successfully with ID {job_id}")
        return {"ID": job_id, "Job Role": job.job_role}

    except Exception as e:
        logger.error("Failed to create job: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail="Error inserting job into
database")
    finally:
        cur.close()
        conn.close()
```

```python
@app.post("/skills-add")
def create_skill(skill: Skill):
    logger.debug(f"Adding skill '{skill.skill_name}' to job {skill.job_id}")
    conn = get_connection()
    if not conn:
        raise HTTPException(status_code=500, detail="Database connection failed")

    try:
        cur = conn.cursor()
        cur.execute(
            '''
            UPDATE "RESUME"."Job_descriptions"
            SET "Skills" = COALESCE("Skills", ARRAY[]::text[]) || %s
            WHERE "ID" = %s
            RETURNING "ID", "Skills";
            ''',
            ([skill.skill_name], skill.job_id)
        )
        updated = cur.fetchone()
        conn.commit()

        if not updated:
            raise HTTPException(status_code=404, detail="Job ID not found")

        logger.info(f"Skill added successfully to job {skill.job_id}.")
        return {"ID": updated[0], "Skills": updated[1]}

    except HTTPException:
        raise
    except Exception as e:
        logger.error("Error adding skill: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail="Failed to update skills")
    finally:
        cur.close()
        conn.close()


def extract_job_role(text: str) -> str:
    """
    Simple heuristic: take the first line or look for "Position" / "Role"
    """
    match = re.search(r"(Position|Role|Job Title)[:\-\s]*(.*)", text,
re.IGNORECASE)
```

```python
    if match:
        return match.group(2).strip()
    # fallback: first non-empty line
    for line in text.split("\n"):
        if line.strip():
            return line.strip()
    return "Unknown Role"

def extract_skills_from_jd(text: str) -> list[str]:
    """
    Extract skills from Job Description text.
    """
    predefined_skills = [
        "Python", "Java", "C++", "SQL", "AWS", "Docker", "Kubernetes",
        "Machine Learning", "Data Analysis", "Excel", "Communication"
    ]
    text_lower = text.lower()
    return [skill for skill in predefined_skills if skill.lower() in text_lower]

 # --------------------------------
# SKILL EXTRACTION
# --------------------------------
def extract_skills_from_jd(text: str) -> list[str]:
    """
    Extract skills dynamically from the JD itself (NO predefined list)
    """
    match = re.search(
        r"(Skills|Requirements|Qualifications)[:\-\n\r]*([\s\S]{0,800})",
        text,
        re.IGNORECASE
    )

    if not match:
        return []

    skills_block = match.group(2)

    # stop if another section starts
    skills_block = re.split(
        r"\n[A-Z][A-Za-z\s]{0,30}[:\n]",
        skills_block
    )[0]
```

```
    skills = [
        s.strip()
        for s in re.split(r",|\n|•|\-|;", skills_block)
        if len(s.strip()) > 2
    ]

    return list(set(skills))




# --------------------------------
# CHUNKING
# --------------------------------
def chunk_text_simple(text: str, chunk_size: int = 500):
    logger.debug(f"Chunking text into chunks of size {chunk_size}")
    words = text.split()
    chunks = [" ".join(words[i:i + chunk_size]) for i in range(0, len(words),
chunk_size)]
    logger.info(f"Generated {len(chunks)} chunks.")
    return chunks
# Dummy pipeline function
# -------------------------------
def process_resume_pipeline(file_path: str, job_description: str, job_id: int) ->
dict:
    """
    Stub for resume processing pipeline.
    Replace this with your actual processing logic.
    """
    logger.debug(f"Processing resume: {file_path} for job {job_id}")

    # Example dummy result
    return {
        "similarity_score": 0.85,
        "match": True,
        "agent_output": "Candidate matches key skills.",
        "chunks_saved": 3,
        "top_chunks": ["chunk1 text", "chunk2 text", "chunk3 text"]
    }

@app.post("/process-resume", response_model=AgentResult)
def process_resume(
    file: UploadFile = File(...),
    job_description: str = Form(...),
```

```
    job_id: int = Form(...)
):
    tmp_dir = tempfile.mkdtemp()
    file_path = os.path.join(tmp_dir, file.filename)

    try:
        # -------------------------
        # Save uploaded file
        # -------------------------
        with open(file_path, "wb") as f:
            f.write(file.file.read())

        # -------------------------
        # Extract text from PDF
        # -------------------------
        resume_text = safe_extract_pdf(file_path)
        if not resume_text.strip():
            raise HTTPException(status_code=400, detail="No text extracted from
resume")

        # -------------------------
        # Extract skills
        # -------------------------
        skills_list = []

        match = re.search(
            r"(Skills|Technical Skills|Key Skills|Professional Skills)[:\-
\n\r]*([\s\S]{0,300})",
            resume_text,
            re.IGNORECASE,
        )
        if match:
            skills_raw = match.group(2)
            # Stop at next header (simple heuristic)
            skills_raw = re.split(r"\n[A-Z][A-Za-z\s]{0,30}[:\n]", skills_raw)[0]
            skills_list = [s.strip() for s in re.split(r",|;|\n|\|", skills_raw)
if s.strip()]

        # -------------------------
        # Chunk text & embed
        # -------------------------
        chunks = chunk_text_simple(resume_text)
        embeddings = [embed_text(chunk).tolist() for chunk in chunks]
```

```
        # -------------------------
        # Save chunks and skills in DB
        # -------------------------
        with get_cursor() as cur:
            for chunk_text, emb in zip(chunks, embeddings):
                cur.execute(
                    '''INSERT INTO "RESUME"."Resume_chunk"
                       ("Job_ID", pdf_chunk, embedding, skills_section)
                       VALUES (%s, %s, %s::vector, %s)''',
                    (job_id, chunk_text, emb, skills_list)  # store skills as
array
                )

        # -------------------------
        # Compare chunks with job description
        # -------------------------
        jd_emb = embed_text(job_description).tolist()
        top_chunks = []
        score = 0

        with get_cursor() as cur:
            cur.execute(
                '''SELECT pdf_chunk, 1-(embedding <=> %s::vector) AS similarity
                   FROM "RESUME"."Resume_chunk"
                   WHERE "Job_ID" = %s
                   ORDER BY similarity DESC
                   LIMIT 5''',
                (jd_emb, job_id)
            )
            rows = cur.fetchall()

        top_chunks = [{"chunk_text": r[0], "similarity": round(r[1]*100, 2)} for
r in rows]
        if top_chunks:
            score = top_chunks[0]["similarity"]

        match_label = (
            "Strong Match" if score > 80 else
            "Partial Match" if score > 50 else
            "Weak Match"
        )
```

```
        # -------------------------
        # Run agent
        # -------------------------
        try:
            agent_output = run_resume_agent(
                resume_text=resume_text,
                job_role="",  # optional: pass job_role if available
                skills=skills_list,
                similarity_score=score,
                match_label=match_label
            )
        except Exception as e:
            logger.error("Agent failed: %s", e, exc_info=True)
            agent_output = f"Agent failed: {str(e)}"

# -------------------------
# Return result
# -------------------------
        return AgentResult(
            filename=file.filename,
            similarity_score=score,
            match=match_label,
            agent_output=agent_output,
            chunks_saved=len(chunks),
            top_chunks=top_chunks,
            skills=skills_list
        )

    except Exception as e:
        logger.error("Error processing resume: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))

    finally:
        shutil.rmtree(tmp_dir, ignore_errors=True)


        #-------------------------
        #compare resume
        #---------------------
@app.post("/compare resumes")
def compare_resumes(files: list[UploadFile] = File(...)):
    if len(files) != 2:
        raise HTTPException(status_code=400, detail="Exactly 2 resumes required")
```

```
    tmp_dirs = [tempfile.mkdtemp() for _ in files]
    file_paths = [os.path.join(tmp_dirs[i], files[i].filename) for i in range(2)]
    texts = []

    try:
        # Save files & extract text
        for i, file in enumerate(files):
            with open(file_paths[i], "wb") as f:
                f.write(file.file.read())

            text = extract_pdf(file_paths[i])
            if not text.strip():
                raise HTTPException(status_code=400, detail=f"No text extracted
from {file.filename}")

            texts.append(text)

        # DB connection
        conn = get_connection()
        cur = conn.cursor()

        # 1 Insert resumes into Resume table if not exists
        for file in files:
            cur.execute(
                '''INSERT INTO "RESUME"."Resume" (filename)
                   VALUES (%s)
                   ON CONFLICT (filename) DO NOTHING''',
                (file.filename,)
            )
        conn.commit()

        # 2 Fetch resume IDs
        def get_resume_id(filename):
            def get_resume_id(filename):
                cur.execute('SELECT id FROM "RESUME"."Resume" WHERE filename
= %s', (filename,))
                row = cur.fetchone()
                if not row:
                    raise HTTPException(status_code=404, detail=f"{filename} not
found in Resume table")
                return row[0]
```

```
        resume1_id = get_resume_id(files[0].filename)
        resume2_id = get_resume_id(files[1].filename)

        # 3 Compute similarity
        emb1 = embed_text(texts[0]).tolist()
        emb2 = embed_text(texts[1]).tolist()
        similarity_score = round(cosine_similarity(emb1, emb2) * 100, 2)

        # 4 Skill overlap (simple word overlap)
        words1 = set(texts[0].lower().split())
        words2 = set(texts[1].lower().split())
        skill_overlap = len(words1 & words2)

        # DB connection
        conn = get_connection()
        cur = conn.cursor()
        # 5 Insert into ResumeComparison table
        cur.execute(
            '''
            INSERT INTO "RESUME"."ResumeComparison"
            ("resume1_id", "resume2_id", "similarity_score", "skill_overlap",
"summary1", "summary2")
            VALUES (%s, %s, %s, %s, %s, %s);
            ''',
            (resume1_id, resume2_id, similarity_score, skill_overlap, texts[0],
texts[1])
        )

        conn.commit()
        return {
            "resume1": files[0].filename,
            "resume2": files[1].filename,
            "similarity_score": similarity_score,
            "skill_overlap": skill_overlap,
            "message": "Comparison saved successfully"
        }

    finally:
        cur.close()
        conn.close()
        for tmp in tmp_dirs:
```

```
            shutil.rmtree(tmp, ignore_errors=True)

# # ---------------------------------
# # AUTO MATCH RESUME
# ---------------------------------
@app.post("/auto-match-resume", response_model=List[AutoMatchResult])
def auto_match_resume(files: List[UploadFile] = File(...)):
    results = []

    for file in files:
        tmp_dir = tempfile.mkdtemp()
        file_path = os.path.join(tmp_dir, file.filename)

        try:
            # -------------------------
            # Save file
            # -------------------------
            with open(file_path, "wb") as f:
                f.write(file.file.read())

            # -------------------------
            # Extract resume text
            # -------------------------
            resume_text = safe_extract_pdf(file_path)
            if not resume_text.strip():
                raise HTTPException(
                    status_code=400,
                    detail=f"No text extracted from {file.filename}"
                )

            # -------------------------
            # Chunk + Embed resume
            # -------------------------
            chunks = chunk_text_simple(resume_text)
            chunk_embeddings = [embed_text(chunk).tolist() for chunk in chunks]

            # -------------------------
            # Fetch jobs
            # -------------------------
            with get_cursor() as cur:
                cur.execute(
                    'SELECT "ID", "Job Role", "Skills" FROM
"RESUME"."Job_descriptions"'
```

```
        )
        jobs = cur.fetchall()

    if not jobs:
        raise HTTPException(
            status_code=404,
            detail="No job descriptions found for auto match"
        )

    # -------------------------
    # Find best job
    # -------------------------
    best_job = None
    best_score = -1
    best_job_text = ""

    for job_id, role, skills in jobs:
        skills_list = skills or []
        jd_text = f"{role}. Required skills: {', '.join(skills_list)}"
        jd_embedding = embed_text(jd_text).tolist()

        scores = [
            cosine_similarity(res_emb, jd_embedding)
            for res_emb in chunk_embeddings
        ]

        avg_score = float(np.mean(scores)) if scores else 0.0

        if avg_score > best_score:
            best_score = avg_score
            best_job = (job_id, role, skills_list)
            best_job_text = jd_text

    if not best_job:
        raise HTTPException(
            status_code=500,
            detail="Failed to determine best job match"
        )

    # -------------------------
    # Final scoring
    # -------------------------
    similarity_score = round(best_score * 100, 2)
```

```python
        match_label = (
            "Strong Match" if similarity_score > 80 else
            "Partial Match" if similarity_score > 50 else
            "Weak Match"
        )

        # -------------------------
        # Run agent
        # -------------------------
        try:
            agent_output = run_resume_agent(
            # resume_text=resume_text,
            job_role=best_job[1],
            skills=best_job[2],
            similarity_score=similarity_score,
            match_label=match_label
        )

        except Exception as e:
            agent_output = f"Agent failed: {str(e)}"

        # -------------------------
        # Store result in DB
        # -------------------------
        try:
            with get_cursor() as cur:
                cur.execute(
                    '''
                    INSERT INTO "RESUME"."AutoMatchResult"
                    (filename, job_id, job_role, skills,
                     similarity_score, match_label, agent_output)
                    VALUES (%s, %s, %s, %s, %s, %s, %s)
                    ''',
                    (
                        file.filename,
                        best_job[0],
                        best_job[1],
                        best_job[2],
                        similarity_score,
                        match_label,
                        agent_output
                    )
```

```
                )
            except Exception:
                logger.error("AutoMatch DB insert failed", exc_info=True)
                raise HTTPException(
                    status_code=500,
                    detail="Failed to store auto match result"
                )

            # -------------------------
            # API response
            # -------------------------
            results.append(
                AutoMatchResult(
                    filename=file.filename,
                    best_match=BestMatch(
                        job_id=best_job[0],
                        job_role=best_job[1],
                        skills=best_job[2]
                    ),
                    similarity_score=similarity_score,
                    match=match_label,
                    agent_output=agent_output

                )
            )

        finally:
            shutil.rmtree(tmp_dir, ignore_errors=True)

    return results

#(I JUST WWANT OT KNOW THAT ITS WORKING AS AGENT OR NOT)
def ocr_pdf(file_path: str) -> str:
    """
    If PDF has no text, use OCR (via pytesseract) on images of each page.
    """
    try:
        pages = convert_from_path(file_path)
        text = ""
        for page in pages:
            text += pytesseract.image_to_string(page) + "\n"
        return text
    except Exception as e:
```

```
        logger.error(f"OCR failed for {file_path}: {e}", exc_info=True)
        return ""

# ----------------------------
# CREATE JOB FROM PDF
# ----------------------------
@app.post("/create-job-from-pdf")
def create_job_from_pdf(file: UploadFile = File(...)):
    tmp_dir = tempfile.mkdtemp()
    file_path = os.path.join(tmp_dir, file.filename)

    try:
        # Save PDF
        with open(file_path, "wb") as f:
            f.write(file.file.read())

        # Extract text
        jd_text = safe_extract_pdf(file_path)
        if not jd_text.strip():
            jd_text = ocr_pdf(file_path)

        if not jd_text.strip():
            raise HTTPException(status_code=400, detail="No text extracted from
JD PDF")

        # Extract job role
        job_role = extract_job_role(jd_text)

        # ☑ Extract skills dynamically (NO predefined list)
        skills = extract_skills_from_jd(jd_text)

        # Save to DB
        with get_cursor() as cur:
            cur.execute(
                '''
                INSERT INTO "RESUME"."Job_descriptions"
                ("Job Role", "Skills", "need_job")
                VALUES (%s, %s, %s)
                RETURNING "ID";
                ''',
                (job_role, skills, jd_text)
            )
            job_id = cur.fetchone()[0]
```

```
        return {
            "message": "Job created successfully from PDF",
            "job_id": job_id,
            "job_role": job_role,
            "skills": skills
        }

    finally:
        shutil.rmtree(tmp_dir, ignore_errors=True)

# @app.post("/create-job-from-pdf")
# def create_job_from_pdf(file: UploadFile = File(...)):
#     tmp_dir = tempfile.mkdtemp()
#     file_path = os.path.join(tmp_dir, file.filename)
#     try:
#         with open(file_path, "wb") as f:
#             f.write(file.file.read())

#         jd_text = safe_extract_pdf(file_path)
#         if not jd_text.strip():
#             jd_text = ocr_pdf(file_path)
#         if not jd_text.strip():
#             raise HTTPException(400, "No text extracted from JD PDF")

#         job_role = extract_job_role(jd_text)
#        skills = extract_skills_from_jd(jd_text)


#         try:
#             agent_output = run_resume_agent(
#                 resume_text=jd_text,
#                 job_role=job_role,
#                 skills=skills,
#                 similarity_score=100,
#                 match_label="Job Description"
#             )
#         except Exception as e:
#             logger.error(f"Agent failed for JD PDF: {e}", exc_info=True)
#             agent_output = f"Agent failed: {str(e)}"

#         with get_cursor() as cur:
#             cur.execute(
```

```
#                 'INSERT INTO "RESUME"."Job_descriptions" ("Job Role", "Skills",
"need_job") VALUES (%s, %s, %s) RETURNING "ID";',
#                 (job_role, skills, jd_text)
#             )
#             job_id = cur.fetchone()[0]

#         return {
#             "message": "Job created successfully from PDF",
#             "job_id": job_id,
#             "job_role": job_role,
#             "skills": skills,
#             "agent_output": agent_output
#         }
#     finally:
#         shutil.rmtree(tmp_dir, ignore_errors=True)

# @app.post("/create-job-from-pdf")
# def create_job_from_pdf(file: UploadFile = File(...)):
#     tmp_dir = tempfile.mkdtemp()
#     file_path = os.path.join(tmp_dir, file.filename)

#     try:
#         # Save PDF
#         with open(file_path, "wb") as f:
#             f.write(file.file.read())

#         # Extract text
#         jd_text = safe_extract_pdf(file_path)
#         if not jd_text.strip():
#             jd_text = ocr_pdf(file_path)
#         if not jd_text.strip():
#             raise HTTPException(400, "No text extracted from JD PDF")

#         # Extract role and skills
#         job_role = extract_job_role(jd_text)
#         skills = extract_skills_from_jd(jd_text)

#         # Run agent
#         try:
#             agent_output = run_resume_agent(
#                 resume_text=jd_text,
#                 job_role=job_role,
#                 skills=skills,
```

```
#                 similarity_score=100,
#                 match_label="Job Description"
#             )
#         except Exception as e:
#             logger.error(f"Agent failed for JD PDF: {e}", exc_info=True)
#             agent_output = f"Agent failed: {str(e)}"

#         # Insert into DB
#         with get_cursor() as cur:
#             cur.execute(
#                 '''
#                 INSERT INTO "RESUME"."Job_descriptions"
#                 ("Job Role", "Skills", "need_job")
#                 VALUES (%s, %s, %s) RETURNING "ID";
#                 ''',
#                 (job_role, skills, jd_text)
#             )
#             job_id = cur.fetchone()[0]

#         return {
#             "message": "Job created successfully from PDF",
#             "job_id": job_id,
#             "job_role": job_role,
#             "skills": skills,
#             "agent_output": agent_output
#         }

#     finally:
#         shutil.rmtree(tmp_dir, ignore_errors=True)
```